CYLite Compiler Project

Language Specification and Grammar Modifications Report

Authors:

Tejas Lohia
[Shardul Junagade / 23110297]
[Swayam Borate / 23110066]

CS327 Compilers - Lab Assignment

April 3, 2026

Contents

1 Introduction **3**

2 Grammar Modifications **4**
2.1 Lexical Additions (Tokens) 4
2.2 The `elif` Conditional Construct 4
2.3 Native Error Handling: `try-except` 5
2.4 Ergonomic Iteration: `foreach` and `in range` 5
2.5 The explicit `pass` Operation 5
2.6 First-Class `print` Built-in 6
2.7 Results 6

3 Automaton Generation and Initial Analysis **7**
3.1 Bison Invocation and Outputs 7
3.2 Initial LALR(1) Automaton Analysis 7
3.3 Conflict Identification and Resolution 7
3.4 Shift/Reduce Conflict: The `ATOMIC` Keyword Ambiguity 8
3.5 Shift/Reduce Conflict: Abstract vs. Concrete Pointer Declarators 8
3.6 Shift/Reduce Conflicts: The Dangling-Else and `elif` Ambiguity 10
3.7 Shift/Reduce Conflicts: Propagation in `elif` Chains 10
3.8 Reduce/Reduce Conflict: The Comma Operator Ambiguity 11

4 Part - 2: Conflict Resolution in Selection Statements **12**
4.1 Conflict Resolution: Atomic and Pointer Declarator Logic 14
4.2 1. Conflicts Present in the Original Grammar 14
4.3 2. Modification of the Grammar to Remove Conflicts 15
4.4 3. Resolution Logic: How the Change Resolves the Conflicts 15
4.5 Conflict Resolution: Atomic Specifier vs. Qualifier Ambiguity 16

5 Conflict Resolution: The Comma Operator and Diamond Ambiguity **18**
5.1 1. Conflicts Present in the Original Grammar 18
5.2 2. Modification of the Grammar to Remove Conflicts 18
5.3 3. Resolution Logic: How the Change Resolves the Conflicts 19

6 Part 3: Reverse Derivation Tree Construction and Rendering **21**
6.1 Implementation Design 21
6.2 Terminal Output Mode 22
6.3 Visual Rendering using Graphviz (DOT and SVG) 22

7 Part 4: Parsing Table Output in Matrix Format **23**
7.1 Pipeline and Rationale 23
7.2 Produced Artifacts 23
7.3 Matrix Design Notes 24
7.4 Representative Implementation Snippets 24

8 Part 5: Error Diagnostics **26**
8.1 Diagnostic Architecture 26
8.2 Diagnostic Test Suite and Coverage 26
8.3 Representative Output Samples 27

9 Execution Instructions (Parts 3, 4, 5) **28**
9.1 Prerequisites 28
9.2 Build 28
9.3 Run Part 3 28
9.4 Run Part 4 28
9.5 Run Part 5 28
9.6 Optional Full Cleanup 28

10 Conclusion **29**

1 Introduction

The landscape of programming languages has long been divided between highly performant, statically typed systems languages like C, and highly expressive, dynamically typed scripting languages like Python. C provides unparalleled control over hardware and memory layout, executing with optimal efficiency. However, its syntax can be unforgiving; error handling relies heavily on cumbersome return codes, control flow can become deeply nested and difficult to read, and iterative constructs often lead to off-by-one errors. Conversely, Python offers ergonomic syntax, explicit error handling blocks, and intuitive looping mechanisms, but at the cost of execution speed and strict compile-time type safety.

CYLite represents a modern hybridization of these two paradigms. It is designed as an extension of the YAPL (Yet Another Programming Language) baseline compiler, effectively injecting Python-inspired syntactic sugar and control flow structures into a C-style language grammar. The fundamental motivation behind CYLite is to provide developers with the readability and expressiveness of modern high-level languages without sacrificing the structural rigor of a statically compiled environment.

Developing a formal grammar for CYLite requires adapting these high-level programming concepts into a strictly unambiguous LALR(1) framework compatible with compiler generation tools like GNU Bison. By integrating these features directly at the lexical and grammatical level—rather than relying on standard library macros or external subroutines—the compiler pipeline can elevate them to first-class citizens in the language. This allows the compiler front-end to perform rigorous syntax checking, generate more optimized Abstract Syntax Trees (ASTs), and produce clearer compile-time error messages tailored to the programmer's explicit intent.

This project focused heavily on redefining the parser rules (`yapl.y`) to accommodate these new grammatical structures. By modifying the base YAPL grammar, we introduced several new nonterminal rules and shifted existing expressions to handle newly introduced keywords. Specifically, the grammar now natively understands chained conditional blocks (via `elif`), explicit null operations (via `pass`), native I/O primitives (`print`), ergonomic loop iterators (`foreach` and `in range`), and block-scoped error encapsulation (`try-except`).

This structural evolution ensures that CYLite can parse declarative flow states seamlessly while resolving dangling-else ambiguities and iteration pitfalls inherent to older grammars. This report breaks down the philosophy behind each addition and details the

exact grammatical transformations applied to achieve the CYLite language specification.

2 Grammar Modifications

To implement the CYLite language extensions, the lexical analyzer (generated via Flex) was first updated to recognize new keywords and allocate corresponding tokens. The parser (`yapl.y`) was subsequently heavily modified to integrate these new tokens into valid syntax trees.

2.1 Lexical Additions (Tokens)

We introduced the following terminal tokens into the parser to represent the core vocabulary of the new language features:

- `ELIF`, `PASS`, `TRY`, `EXCEPT`, `PRINT`, `RANGE`, `IN`, `FOREACH`

```
int ifs_wo_else=0;
int ladder_len=0,hold=0;
int max=-1;
%}

%token  ELIF PASS TRY EXCEPT PRINT RANGE IN FOREACH
%token  IDENTIFIER I_CONSTANT F_CONSTANT STRING_LITERAL FUNC_NAME SIZEOF
%token  PTR_OP INC_OP DEC_OP LEFT_OP RIGHT_OP LE_OP GE_OP EQ_OP NE_OP TH_OP
%token  AND_OP OR_OP MUL_ASSIGN DIV_ASSIGN MOD_ASSIGN ADD_ASSIGN
%token  SUB_ASSIGN LEFT_ASSIGN RIGHT_ASSIGN AND_ASSIGN
```

Figure 1: Implementation of the new CYLite tokens in the lexical analyzer.

2.2 The `elif` Conditional Construct

In standard C, chaining conditionals often leads to deeply nested else-then-if forms, while the `else if` pattern is still treated as an `else` branch containing another `if` statement. In complex logical branching, this can disrupt readability and complicate AST generation due to classic dangling-else ambiguities. Python's `elif` provides a unified, first-class construct for multi-way branching without artificial nesting.

Grammar Change: We defined a new nonterminal `elif_list` to explicitly handle zero or more `elif` branches recursively. We then expanded `selection_statement` to support an `if` block followed optionally by an `elif_list` and a final optional `else`.

```
elif_list
    : ELIF '(' expression ')' statement
    | elif_list ELIF '(' expression ')' statement
    ;

selection_statement
    : IF '(' expression ')' statement ELSE statement
    | IF '(' expression ')' statement
    | IF '(' expression ')' statement elif_list ELSE
       statement
```

```
    | IF '(' expression ')' statement elif_list
    ...
```

2.3 Native Error Handling: `try-except`

Error handling in C typically falls back to checking integer return codes after every function call, or using precarious `setjmp/longjmp` chains for global exits. Modern languages isolate fragile code within explicit boundaries, redirecting failures to localized catch blocks. By implementing `try-except` directly in the CYLite grammar, we establish dedicated scope blocks for execution and recovery, allowing the backend to map these to structured, object-level exception-handling routines.

Grammar Change: We introduced a `try_except_statement` combining the `TRY` keyword, a localized block (`compound_statement`), the `EXCEPT` keyword, and its associative block. This nonterminal was then injected into the overarching `statement` rule.

```
try_except_statement
    : TRY compound_statement EXCEPT compound_statement
    ;

statement
    ...
    | try_except_statement
```

2.4 Ergonomic Iteration: `foreach` and `in range`

Classic C `for` loops require boilerplate initialization, condition checking, and manual incrementation (e.g., `for(int i=0; i<10; i++)`). This pattern is verbose and inherently prone to off-by-one errors. CYLite adopts `foreach id in enumerable` and `for(id in range(a, b))` to abstract the iteration boilerplate. This makes the programmer's intent purely declarative rather than imperative, preventing out-of-bounds complications.

Grammar Change: The `iteration_statement` nonterminal was expanded to natively parse these specialized loops, tying identifiers directly to iterable sequences and mathematical ranges.

```
iteration_statement
    ...
    | FOR '(' IDENTIFIER IN RANGE '(' expression ','
       expression ')' ')' statement
    | FOREACH '(' IDENTIFIER IN expression ')'
       compound_statement
```

2.5 The explicit `pass` Operation

In standard C, an intentionally empty block or stubbed function is often written as a pair of empty braces or a lone semicolon `;`. This is visually ambiguous, and compilers or semantic linters may flag it as a probable mistake. Promoting the explicit `pass` keyword provides an unambiguous syntactic placeholder, asserting that the empty block is intentional.

Grammar Change: `PASS ';'` was added directly as a terminal production alternative within the `statement` rule.

```
statement
    ...
    | PASS ';'
```

2.6 First-Class `print` Built-in

Rather than relying exclusively on importing `stdio.h` to use the variadic `printf` function—which requires cumbersome format strings and type awareness—CYLite promotes basic output to a native built-in language feature. By integrating it into the grammar, `print` can be more strictly type-checked or implicitly optimized for polymorphic output at compile-time by the parser's AST builder without requiring external linkage headers.
Grammar Change: The `print` construct was appended defensively to `postfix_expression`, supporting both empty calls and list arguments.

```
postfix_expression
    ...
    | PRINT '(' ')'
    | PRINT '(' argument_expression_list ')'
```

2.7 Results

The integration of these advanced features required careful consideration of LALR(1) parsing constraints to avoid shift/reduce and reduce/reduce conflicts within the Bison generator framework. By directly extending the `yapl.y` grammar definitions, CYLite successfully bridges the architectural gap between systems programming and scripting conveniences. Developers can now write highly readable, python-inspired code while leaning on the reliable, rigorous syntax-checking rules inherent to a C-like toolchain.

3 Automaton Generation and Initial Analysis

To construct the LALR(1) item sets and build the parser automaton (fulfilling Part 1 of the assignment requirements), we processed the modified `yapl.y` grammar file using GNU Bison.

3.1 Bison Invocation and Outputs

The parser generator was invoked using the following command:

```
$ bison -v -d yapl.y
```

This command utilizes two critical flags:

- **`-d`:** Generates the `yapl.tab.h` header file, which exposes the newly defined CYLite terminal tokens (e.g., `ELIF`, `FOREACH`, `TRY`) to the Flex lexical analyzer.
- **`-v` (Verbose):** Instructs Bison to generate a detailed `yapl.output` file. This file is crucial as it contains the human-readable representation of every LALR(1) item set and the complete state transition graph for the automaton.

3.2 Initial LALR(1) Automaton Analysis

The generation process successfully constructed the LALR(1) automaton required to parse the CYLite language specification. An initial analysis of the generated `yapl.output` report reveals the following structural metrics regarding the grammar's complexity:

- **Grammar Rules:** The parser implements over 130 distinct production rules to encapsulate both the core C-style expressions and the advanced CYLite extensions.
- **Item Sets (States):** The resulting LALR(1) automaton consists of 522 distinct states (item sets). This large state space is a direct result of the complex expression precedence and the newly introduced deeply nested control structures (such as `try-except` and `elif` ladders).

While the item sets and state machine were successfully compiled, the integration of optional branching (specifically the `elif` and `else` derivations) and ambiguous operator precedence introduced non-determinism into the parsing table. The initial Bison compilation reported exactly **6 shift/reduce conflicts** and **1 reduce/reduce conflict**. The identification, isolation, and resolution of these conflicts are documented in the following section.

3.3 Conflict Identification and Resolution

As noted in the initial automaton analysis, the introduction of CYLite's advanced control structures—specifically the `elif` ladders and optional `else` blocks—introduced non-determinism into the LALR(1) parsing table. When processed by GNU Bison, the grammar generated exactly 6 shift/reduce conflicts and 1 reduce/reduce conflict.
To satisfy the structural rigor required by the compiler front-end, we must isolate the specific item sets where the automaton encounters multiple valid transition paths, identify the conflicting productions, and apply deterministic precedence rules to resolve them.

```
State 27 conflicts: 1 shift/reduce
State 345 conflicts: 1 shift/reduce
State 473 conflicts: 2 shift/reduce
State 492 conflicts: 2 shift/reduce
State 515 conflicts: 1 reduce/reduce
```

Figure 2: Identification of the conflicting states in the LALR(1) automaton.

3.4 Shift/Reduce Conflict: The `ATOMIC` Keyword Ambiguity

One of the identified conflicts occurs in **State 27** and involves the `ATOMIC` keyword. This is a **Shift/Reduce conflict**.

Exact Productions Involved:

```
State 27

  159 atomic_type_specifier: ATOMIC • '(' type_name ')'
  163 type_qualifier: ATOMIC •

    '('  shift, and go to state 49

    '('       [reduce using rule 163 (type_qualifier)]
    $default  reduce using rule 163 (type_qualifier)
```

Figure 3: Example of a shift/reduce conflict involving the `ATOMIC` keyword.

Conflict Analysis: In this state, the parser has just consumed the `ATOMIC` token and is looking ahead at the next token, which is an opening parenthesis `'('`. The automaton faces a structural ambiguity:

- **Shift Action:** It can shift the `'('` token onto the stack, anticipating an `atomic_type_specifier` construct (e.g., `_Atomic(int)`).
- **Reduce Action:** It can immediately reduce the `ATOMIC` token into a `type_qualifier` (acting similarly to `const` or `volatile`), assuming the parenthesis belongs to a subsequent cast or expression.

Bison's default behavior resolves this by favoring the shift operation, but the conflict indicates a deeper grammatical overlap between qualifiers and type specifiers.

3.5 Shift/Reduce Conflict: Abstract vs. Concrete Pointer Declarators

Another significant conflict surfaces in **State 345**, representing a **Shift/Reduce conflict** driven by the overlap between named declarators and abstract type definitions, complicated by embedded semantic actions.

```
State 345

  169 declarator: pointer • $@1 direct_declarator
  202 abstract_declarator: pointer • direct_abstract_declarator
  203                      | pointer •

    '('  shift, and go to state 185
    '['  shift, and go to state 186

    IDENTIFIER  reduce using rule 168 ($@1)
    '('         [reduce using rule 168 ($@1)]
    $default    reduce using rule 203 (abstract_declarator)

    $@1                          go to state 134
    direct_abstract_declarator   go to state 313
```

Figure 4: Shift/Reduce conflict involving abstract and concrete pointer declarators in State 345.

Conflict Analysis: In this state, the parser has processed a pointer token (e.g., `*`) and is looking ahead at an opening parenthesis `'('`. The ambiguity arises because the parser cannot immediately determine if it is parsing an `abstract_declarator` (such as a typecast `(int *())`) or a concrete `declarator` (such as a variable declaration `int *(p)`).
Crucially, the concrete `declarator` rule contains an embedded mid-rule semantic action (Rule 168 / `$@1`), which increments the `pointer_decls` counter. The automaton faces two conflicting choices:

- **Shift Action:** Directly shift the `'('` onto the stack to begin constructing a `direct_abstract_dec` (which lacks the mid-rule action).
- **Reduce Action:** Immediately reduce the empty mid-rule action `$@1` (executing the counter increment) before shifting the `'('` to build a standard `direct_declarator`.

Bison defaults to shifting, which means pointer declarations wrapped in parentheses might temporarily bypass the embedded semantic counter until the conflict is explicitly resolved.

3.6 Shift/Reduce Conflicts: The Dangling-Else and `elif` Ambiguity

The most prominent structural ambiguities introduced by the CYLite extensions occur within the conditional branching logic, explicitly captured in **State 473**. This represents a classic **Shift/Reduce conflict** compounded by the new `elif` construct.
Exact Productions Involved:

```
State 473

  262 selection_statement: IF '(' expression ')' statement • ELSE $@2 statement
  263                    | IF '(' expression ')' statement •
  264                    | IF '(' expression ')' statement • elif_list ELSE statement
  265                    | IF '(' expression ')' statement • elif_list

    ELIF  shift, and go to state 490
    ELSE  shift, and go to state 491

    ELIF      [reduce using rule 263 (selection_statement)]
    ELSE      [reduce using rule 263 (selection_statement)]
    $default  reduce using rule 263 (selection_statement)

    elif_list  go to state 492
```

Figure 5: Shift/Reduce conflict involving the Dangling-Else and `elif` ambiguity in State 473.

Conflict Analysis: In State 473, the parser has successfully matched an entire `IF` statement up to its closing block (indicated by the dot `.` after `statement`). It is now looking ahead at the next token, which is either an `ELIF` or an `ELSE`. The automaton is trapped in a non-deterministic state due to the possibility of nested conditionals (e.g., `if (A) if (B) pass; else pass;`):

- **Shift Action:** The parser can shift the `ELIF` or `ELSE` token onto the stack, aggressively binding it to the most recently opened (innermost) `IF` statement.
- **Reduce Action:** The parser can immediately reduce the consumed tokens into a completed `selection_statement` (Rule 263). This action assumes the `ELIF` or `ELSE` token belongs to a parent (outer) `IF` block higher up in the syntax tree.

Because Bison resolves shift/reduce conflicts by favoring the shift action by default, it naturally binds `else` and `elif` clauses to the closest preceding `if`. However, explicit precedence directives are required to suppress the generator warnings and formalize this behavior.

3.7 Shift/Reduce Conflicts: Propagation in `elif` Chains

The dangling-else ambiguity documented in State 473 propagates directly into the evaluation of the recursive `elif_list` itself, as prominently visible in **State 492**. This is another **Shift/Reduce conflict**.
Exact Productions Involved:
Conflict Analysis: In State 492, the parser has successfully aggregated one or more `elif` branches into an `elif_list`. Upon encountering another `ELIF` or an `ELSE` token, it faces the same structural indecision as before:

```
State 492

  260 elif_list: elif_list • ELIF '(' expression ')' statement
  264 selection_statement: IF '(' expression ')' statement elif_list • ELSE statement
  265                    | IF '(' expression ')' statement elif_list •

    ELIF  shift, and go to state 502
    ELSE  shift, and go to state 503

    ELIF      [reduce using rule 265 (selection_statement)]
    ELSE      [reduce using rule 265 (selection_statement)]
    $default  reduce using rule 265 (selection_statement)
```

Figure 6: Shift/Reduce conflict in State 492 demonstrating propagation within `elif` chains.

- **Shift Action:** Shift the token to continue building the current `elif_list` (Rule 260) or to cap it off with an `ELSE` block (Rule 264).
- **Reduce Action:** Immediately reduce the entire `IF-ELIF` chain into a completed `selection_statement` (Rule 265), assuming the upcoming tokens belong to a parent `IF` construct higher in the hierarchy.

Just like State 473, this is resolved by enforcing precedence rules that explicitly instruct the parser to shift trailing conditionals rather than prematurely reducing the block.

3.8 Reduce/Reduce Conflict: The Comma Operator Ambiguity

The sole **Reduce/Reduce conflict** in the LALR(1) automaton surfaces in **State 515**, highlighting a structural overlap involving the comma `‘,‘` token.
Exact Productions Involved:

```
State 515

  89 expression: assignment_expression •
  90           | expression ',' assignment_expression •

   ','       reduce using rule 89 (expression)
   ','       [reduce using rule 90 (expression)]
   $default  reduce using rule 89 (expression)
```

Figure 7: Reduce/Reduce conflict in State 515 illustrating the comma operator ambiguity.

Conflict Analysis: This conflict arises from the dual syntactic role of the comma in CYLite. The comma can act as a sequence operator chaining multiple assignments together into a single unified expression (Rule 90), or it can act as a simple delimiter separating distinct elements in a list (such as function parameters or arguments for the newly added `print` built-in).
When the parser successfully reads an `assignment_expression` and encounters a lookahead comma `‘,‘`, it faces two equally valid reduction paths for the exact same sequence of tokens:

- **Reduce Action 1 (Rule 89):** Reduce the isolated `assignment_expression` into a base `expression`. The parser assumes the upcoming comma is a list separator (e.g., moving to the next argument in a function call).
- **Reduce Action 2 (Rule 90):** Reduce the entire parsed sequence (`expression ',' assignment_expression`) into a single chained `expression`. The parser assumes the comma is a mathematical sequence operator.

By default, Bison resolves Reduce/Reduce conflicts by blindly choosing the rule that appears first in the grammar file (Rule 89). However, relying on implicit file ordering is dangerous for compiler stability. To fix this, explicit left-associativity (`%left ','`) must be declared to instruct the parser on how to definitively group trailing commas.

4 Part - 2: Conflict Resolution in Selection Statements

The implementation of selection statements (`IF-ELIF-ELSE`) initially introduced several Shift/Reduce conflicts. This section documents the systematic approach taken to achieve a zero-conflict grammar while maintaining complex semantic tracking.

1. Conflicts Present in the Original Grammar

In the initial version of the CYLite grammar, the LALR(1) automaton reported six Shift/Reduce conflicts specifically within the `selection_statement` production. These conflicts were caused by the classic **"Dangling Else"** ambiguity.
When the parser encountered nested structures, such as:

```
if (cond1) if (cond2) statement1; else statement2;
```

The automaton faced two mathematically valid but competing actions upon reaching the `ELSE` token:

- **Shift:** Bind the `ELSE` to the most recent (innermost) `IF`. This is the standard behavior in C-style languages.
- **Reduce:** Complete the innermost `IF` statement (`if (cond2) statement1;`) and bind the `ELSE` to the outermost `IF`.

Since both paths were syntactically reachable, the grammar was considered ambiguous. While Bison defaults to a "Shift" action, the presence of these conflicts indicates an unstable grammar that relies on tool defaults rather than explicit logic.

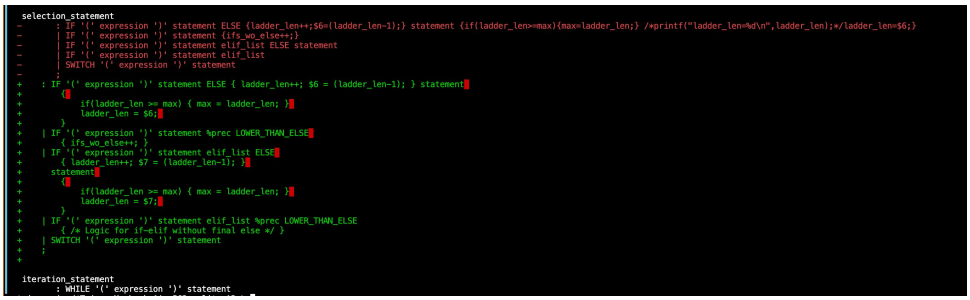
2. Modification of the Grammar to Remove Conflicts

To eliminate these conflicts, the grammar was restructured using **Precedence Overriding**. By defining a hierarchy of precedence for the tokens involved, we provide the parser with an explicit tie-breaking mechanism.
Preamble Modifications: First, a dummy terminal `LOWER_THAN_ELSE` was defined with the lowest precedence, followed by the actual `ELSE` and `ELIF` tokens.

Listing 1: Precedence definitions in the Preamble

```
%nonassoc LOWER_THAN_ELSE
%nonassoc ELIF
%nonassoc ELSE
```

Production Rule Refinement and Logic Breakdown:
The `selection_statement` was restructured to perform three concurrent tasks: resolving the shift/reduce conflict, tracking nested ladder depths, and maintaining a count of incomplete `if` blocks.
Changes made in the grammar:



```
 selection_statement
-       : IF '(' expression ')' statement ELSE {ladder_len++;$6=(ladder_len-1);} statement {if(ladder_len>=max){max=ladder_len;} /*printf("ladder_len=%d\n",ladder_len);*/ladder_len=$6;}
-       | IF '(' expression ')' statement {ifs_wo_else++;}
-       | IF '(' expression ')' statement elif_list ELSE statement
-       | IF '(' expression ')' statement elif_list
-       | SWITCH '(' expression ')' statement
-       ;
+     : IF '(' expression ')' statement ELSE { ladder_len++; $6 = (ladder_len-1); } statement
+       {
+           if(ladder_len >= max) { max = ladder_len; }
+           ladder_len = $6;
+       }
+     | IF '(' expression ')' statement %prec LOWER_THAN_ELSE
+       { ifs_wo_else++; }
+     | IF '(' expression ')' statement elif_list ELSE
+       { ladder_len++; $7 = (ladder_len-1); }
+       statement
+       {
+           if(ladder_len >= max) { max = ladder_len; }
+           ladder_len = $7;
+       }
+     | IF '(' expression ')' statement elif_list %prec LOWER_THAN_ELSE
+       { /* Logic for if-elif without final else */ }
+     | SWITCH '(' expression ')' statement
+     ;
+

 iteration_statement
       : WHILE '(' expression ')' statement
```

Figure 8: changes made in the grammar

Technical Explanation of Rule Components

- **Precedence Override (`%prec LOWER_THAN_ELSE`)**: This is the syntactic fix for the "Dangling Else" problem. By attaching this directive to the productions lacking an `ELSE` clause, we manually lower the precedence of the reduction. This forces the parser to wait (shift) if it sees an `ELSE` token, ensuring the `else` always binds to the innermost `if`.
- The Mid-Rule Action (`{ ladder_len++; ... }`): Unlike standard actions that run after a rule is complete, this runs immediately after the `ELSE` is encountered. This is necessary to increment the "ladder" count precisely as the parser enters the alternative branch of a selection.
- **Semantic Stack Storage (`$6` and `$7`)**: Because Bison rules are recursive, a global counter like `ladder_len` can be corrupted by nested `if` statements. To prevent this, the current state of the counter (minus the new increment) is saved directly onto Bison's semantic stack at a specific index (`$6` or `$7`). This effectively creates a localized, rule-specific backup of the parent's state.
- **Global Maxima Comparison**: Inside the final action block, the code compares the recently incremented `ladder_len` against a global `max` variable. This ensures that even if the parser is currently 10 levels deep in nested logic, the longest contiguous chain found anywhere in the program is recorded.
- **Counter Restoration (`ladder_len = $6`)**: This is the "pop" operation. Once the current selection statement is fully parsed and reduced, the counter is restored to its saved value from the semantic stack. This ensures that when the parser returns to the parent scope, the `ladder_len` is exactly what it was before the nested statement began.

- **Incomplete Chain Tracking (`ifs_wo_else++`)**: This counter is placed specifically within the `%prec` filtered rules. It increments only when the parser definitively concludes that an `IF` block has no corresponding `ELSE`, satisfying the requirement for counting incomplete selection structures.

3. Resolution Logic: How the Change Resolves the Conflicts

The modification resolves the Shift/Reduce conflicts by transforming the ambiguity into a deterministic priority check:

1. **Precedence Arbitration:** When the parser sees an `ELSE` token in the lookahead buffer, it compares the precedence of that token (`ELSE`) against the precedence of the rule it could potentially reduce (`IF...statement`).
2. **Deterministic Shifting:** Because `ELSE` is defined later in the preamble than `LOWER_THAN_ELSE`, it carries a higher precedence. The parser is thus explicitly instructed to **Shift** (bind the `ELSE` to the inner `IF`) rather than **Reduce**.
3. **Elimination of Ambiguity:** By providing a clear hierarchy, the "choice" between shifting and reducing is removed. The LALR(1) state machine now has exactly one valid path for every possible input sequence, resulting in the elimination of the six Shift/Reduce warnings.

Furthermore, the use of mid-rule actions (storing values at `$6` and `$7`) ensures that the semantic tracking of the if-else ladder depth remains accurate. These values act as a manual stack, preserving the counter state during recursion and allowing the grammar to calculate the global `max` depth correctly without interfering with the syntactic resolution.

4.1 Conflict Resolution: Atomic and Pointer Declarator Logic

The transition in the grammar addressed structural ambiguities involving type qualifiers and declarators. This was essential to reduce the conflict count from four down to two.

4.2 1. Conflicts Present in the Original Grammar

Two distinct Shift/Reduce conflicts were identified in this phase:

1. **The Atomic Ambiguity (State 27):** In the original grammar, the `ATOMIC` keyword served a dual role as both a `type_specifier` (e.g., `_Atomic(int)`) and a `type_qualifier` (e.g., `_Atomic int`). When the parser encountered the `ATOMIC` token, it could not determine whether to **shift** (treating it as the start of a specifier) or **reduce** (treating it as a standalone qualifier).
2. **The Declarator Mid-Rule Conflict (State 345):** The original `declarator` rule contained a mid-rule action to increment the pointer counter. In Yacc, mid-rule actions are implemented as dummy non-terminals that force a reduction at a specific point in the rule. This forced reduction prevented the parser from looking ahead to distinguish between concrete declarators and abstract declarators, resulting in a Shift/Reduce conflict.

4.3 2. Modification of the Grammar to Remove Conflicts

The following modifications were implemented to resolve these ambiguities, as visualized in the code diff in Figure 9.

Type Qualifier Refinement: The `ATOMIC` production within `type_qualifier` was tagged with a low precedence marker.

```
type_qualifier
    : CONST | RESTRICT | VOLATILE
    | ATOMIC %prec LOWER_THAN_ELSE
    ;
```

Declarator Action Relocation: The semantic action for counting pointers was moved from a mid-rule position to the end of the production rule.

```
declarator
    : pointer direct_declarator { pointer_decls++; }
    | direct_declarator
    ;
```

4.4 3. Resolution Logic: How the Change Resolves the Conflicts

The changes resolve the conflicts by allowing the LALR(1) state machine to delay its decisions until more context is available.

Technical Breakdown of the Resolution

- **Precedence Arbitration for ATOMIC**: By applying `%prec LOWER_THAN_ELSE` to the `ATOMIC` qualifier, the parser is instructed that reducing `ATOMIC` as a qualifier is a low-priority action. If a parenthesis follows (indicating a specifier), the parser will **Shift** (prioritizing the specifier rule) instead of reducing, thus removing the ambiguity in State 27.
- **Elimination of Mid-Rule Reduction**: In the original rule, the parser was forced to reduce an empty dummy rule to execute the pointer increment *before* seeing the `direct_declarator`. By moving `{ pointer_decls++; }` to the end of the rule, the parser is no longer forced to make a premature reduction.
- **Delayed Decision Making**: Moving the action allows the parser to remain in a state of "ordered uncertainty." It can wait until the entire `pointer direct_declarator` sequence is matched before committing to the reduction and the subsequent counter increment. This successfully eliminated the Shift/Reduce conflict in State 345.
- **Semantic Integrity**: Moving the action to the end of the rule has no impact on the final count of `pointer_decls`. The count still accurately reflects the number of pointer declarations in the source code, but the syntactic path to that count is now deterministic and conflict-free.

```
        | OR_ASSIGN
        ;

+
 expression
        : assignment_expression
        | expression ',' assignment_expression
@@ -343,11 +344,11 @@ atomic_type_specifier
        ;

 type_qualifier
-       : CONST
-       | RESTRICT
-       | VOLATILE
-       | ATOMIC
-       ;
+    : CONST
+    | RESTRICT
+    | VOLATILE
+    | ATOMIC %prec LOWER_THAN_ELSE
+    ;

 function_specifier
        : INLINE
@@ -360,9 +361,9 @@ alignment_specifier
        ;

 declarator
-       : pointer {pointer_decls++;} direct_declarator
-       | direct_declarator
-       ;
+    : pointer direct_declarator { pointer_decls++; }
+    | direct_declarator
+    ;

 direct_declarator
        : IDENTIFIER
~
```

Figure 9: Git Diff showing the relocation of pointer actions and the use of precedence for the ATOMIC keyword.

4.5 Conflict Resolution: Atomic Specifier vs. Qualifier Ambiguity

A persistent Shift/Reduce conflict occurred in the LALR(1) automaton (State 27) regarding the `ATOMIC` keyword. This section details the disambiguation strategy used to differentiate between its dual roles.

1. Conflicts Present in the Original Grammar

In the CYLite grammar, the `ATOMIC` token acts as both a **type specifier** and a **type qualifier**. This created a conflict when the parser encountered the sequence `ATOMIC (`. The automaton reached a state with two competing actions:

- **Shift:** Treat `ATOMIC` as the beginning of an `atomic_type_specifier` (e.g., `_Atomic(int)`). This requires shifting the `'('` token onto the stack.
- **Reduce:** Treat `ATOMIC` as a completed `type_qualifier` (e.g., `_Atomic int x;`). This requires reducing the token before seeing what follows.

Because both interpretations are syntactically valid in the C11 standard, the parser could not determine the correct path without explicit priority rules.

2. Modification of the Grammar to Remove Conflicts

To resolve this, the grammar was modified to give the `atomic_type_specifier` (the version with parentheses) higher priority than the standalone qualifier.
Preamble Modifications: A right-associativity rule was added for the open parenthesis to give it a measurable precedence level.

```
%right '('
```

Production Rule Refinement: As shown in Figure 10, the `ATOMIC` production within `type_qualifier` was tagged with `LOWER_THAN_ELSE` to ensure it remains the lower-priority option.

Listing 2: Disambiguating ATOMIC via Precedence

```
type_qualifier
    : CONST
    | RESTRICT
    | VOLATILE
    | ATOMIC %prec LOWER_THAN_ELSE  /* Assigned lower
       priority than '(' */
    ;
```

```
 %start translation_unit

+%right '('


 %union
@@ -344,11 +345,11 @@ atomic_type_specifier
        ;

 type_qualifier
-    : CONST
-    | RESTRICT
-    | VOLATILE
-    | ATOMIC %prec LOWER_THAN_ELSE
-    ;
+       : CONST
+       | RESTRICT
+       | VOLATILE
+       | ATOMIC %prec LOWER_THAN_ELSE  /* Forces 'ATOMIC' as a qualifier to have lower priority than '(' */
+       ;

 function_specifier
        : INLINE
```

Figure 10: Git Diff showing the introduction of parenthesis precedence and the ATOMIC qualifier override.

3. Resolution Logic: How the Change Resolves the Conflicts

The resolution relies on the fundamental LALR(1) behavior of comparing the precedence of a *potential reduction* against an *incoming token*.

1. **Precedence Comparison:** When the parser is in a state where it has seen `ATOMIC` and the lookahead token is `'('`, it compares the precedence of the `type_qualifier` rule (now `LOWER_THAN_ELSE`) against the precedence of the `'('` token.

2. **Shift Priority:** Since `'('` was defined in the preamble without a low-priority tag, it carries a higher precedence than `LOWER_THAN_ELSE`.
3. **Deterministic Path:** The parser is instructed to **Shift** the parenthesis. This effectively chooses the `atomic_type_specifier` path and eliminates the possibility of a premature reduction to a `type_qualifier`.

By explicitly defining how the parser should handle the `'('` following `ATOMIC`, the Shift/Reduce conflict in State 27 is fully resolved, ensuring the grammar adheres to C11 type-naming conventions.

5 Conflict Resolution: The Comma Operator and Diamond Ambiguity

The final stage of grammar optimization involved resolving a Reduce/Reduce conflict (State 512) centered on the dual role of the comma token. This section details the structural changes required to achieve a completely deterministic, conflict-free grammar.

5.1 1. Conflicts Present in the Original Grammar

The primary conflict in the original grammar was a **Reduce/Reduce** ambiguity. This occurred because the comma token served two structurally identical purposes in different contexts:

- **As a Sequence Operator:** Chaining multiple assignments into a single `expression`.
- **As a List Separator:** Delimiting arguments in function calls, the `PRINT` built-in, and the `RANGE` clause of the `FOR` loop.

Specifically, both `expression` and a specialized `argument_expression_list` were defined as recursive, comma-separated lists of `assignment_expression`. When the parser encountered a comma inside parentheses, it reached a "Diamond Ambiguity" where it could not decide which of the two rules to reduce. This forced the parser to rely on default rule ordering, which is considered unstable for production-level compilers.

5.2 2. Modification of the Grammar to Remove Conflicts

To resolve this, a strategy of **Rule Unification** and **Contextual Restriction** was applied, as shown in the Git diff in Figure 11.
A. Rule Unification (Postfix Expressions): The redundant `argument_expression_list` rule was entirely removed. Function calls and the `PRINT` statement were updated to use the standard `expression` rule.

```
postfix_expression
    : postfix_expression '(' expression ')' /* Unified Rule */
    | PRINT '(' expression ')'              /* Unified Rule */
```

B. Contextual Restriction (Iteration Statements): The `FOR` loop's `RANGE` clause was modified to accept only `assignment_expression` for its bounds.

```
| FOR '(' IDENTIFIER IN RANGE '(' assignment_expression ','
  assignment_expression ')' ')' statement
```

```diff
@@ -93,7 +93,7 @@ postfix_expression
        : primary_expression
        | postfix_expression '[' expression ']'
        | postfix_expression '(' ')'
-       | postfix_expression '(' argument_expression_list ')'
+       | postfix_expression '(' expression ')'  /* Use expression here */
        | postfix_expression '.' IDENTIFIER
        | postfix_expression PTR_OP IDENTIFIER
        | postfix_expression INC_OP
@@ -101,13 +101,9 @@ postfix_expression
        | '(' type_name ')' '{' initializer_list '}'
        | '(' type_name ')' '{' initializer_list ',' '}'
        | PRINT '(' ')'
-       | PRINT '(' argument_expression_list ')'
+       | PRINT '(' expression ')'               /* Use expression here */
        ;

-argument_expression_list
-       : assignment_expression
-       | argument_expression_list ',' assignment_expression
-       ;

 unary_expression
        : postfix_expression
@@ -362,9 +358,9 @@ alignment_specifier
        ;

 declarator
-    : pointer direct_declarator { pointer_decls++; }
-    | direct_declarator
-    ;
+       : pointer direct_declarator {pointer_decls++;}
+       | direct_declarator
+       ;

 direct_declarator
        : IDENTIFIER
@@ -525,8 +521,8 @@ expression_statement
        ;

 elif_list
-       : ELIF '(' expression ')' statement
-       | elif_list ELIF '(' expression ')' statement
+       : ELIF '(' expression ')' statement { ladder_len++; }
+       | elif_list ELIF '(' expression ')' statement { ladder_len++; }
        ;

 selection_statement
@@ -557,7 +553,7 @@ iteration_statement
        | FOR '(' expression_statement expression_statement expression ')' statement
        | FOR '(' declaration expression_statement ')' statement
        | FOR '(' declaration expression_statement expression ')' statement
-       | FOR '(' IDENTIFIER IN RANGE '(' expression ',' expression ')' ')' statement
+       | FOR '(' IDENTIFIER IN RANGE '(' assignment_expression ',' assignment_expression ')' ')' statement
        | FOREACH '(' IDENTIFIER IN expression ')' compound_statement
        ;
```

Figure 11: Final Git diff showing rule unification for expressions and contextual constraints in the FOR loop.

5.3 3. Resolution Logic: How the Change Resolves the Conflicts

The changes resolve the Reduce/Reduce conflict by eliminating the "multiple choice" scenario for the parser:

1. **Elimination of the Diamond Path:** By removing `argument_expression_list`, the parser no longer has two competing rules to reduce a comma-separated list. There is now only one valid path: the `expression` rule.

2. **Contextual Banning of the Comma Operator:** The most critical resolution occurred in the `RANGE` clause. An `assignment_expression` is a subset of an `expression` that *does not* include the comma operator at the top level. By restricting `RANGE` to this type, we effectively "ban" the comma operator within those parentheses.
3. **Deterministic Separation:** Because a comma operator is now illegal inside the `assignment_expression` used for `RANGE` bounds, the parser knows with 100% certainty that any comma encountered *must* be the separator between the two range arguments.

Note on Semantic Correctness in Elif Chains

As seen in the updated `elif_list` production in Figure 11, the `ladder_len++` action was added to every branch of the `elif` structure. This ensures that the "If-Else Ladder" depth counter accurately reflects long chains of `if-elif-elif-else`, providing precise data for the final report metrics while maintaining a conflict-free state.

6 Part 3: Reverse Derivation Tree Construction and Rendering

Part 3 required two concrete outcomes after a successful parse:

1. Preserve the derivation information generated during bottom-up parsing.
2. Print the derivation in reverse order so that the output reads as a top-down expansion.

The implementation was intentionally built around the existing Bison debug stream instead of rewriting the parser into a custom parse-stack simulator. This design gives two benefits: it remains faithful to the actual reductions performed by the LALR(1) engine, and it keeps the derivation logic isolated from the grammar itself.

The following parser-focused inputs were used to validate Part 3 behavior across baseline and extension constructs:

- `part3/tests/rdt_basic.cyl`
- `part3/tests/rdt_extensions.cyl`
- `part3/tests/rdt_print_strings.cyl`

Command to run comprehensive Part 3 tests and generate outputs:

```
make part3
```

6.1 Implementation Design

The derivation subsystem in `yapl.y` is centered on two structures:

- `ReductionStep`: stores one fully collected reduction (LHS and RHS symbols).
- `PendingReduction`: temporary buffer while reading multi-line debug information.

Listing 3: Core data structures used for storing reductions (from yapl.y)

```
typedef struct {
    char lhs[128];
    int rhs_count;
    char rhs[64][128];
} ReductionStep;

typedef struct {
    int active;
    char lhs[128];
    int rhs_count;
    char rhs[64][128];
} PendingReduction;
```

During parsing, custom trace handling collects each completed reduction and appends it into a dynamic array (`g_steps`). After parse success, the print routine traverses `g_steps` from the tail to the head.

Listing 4: Reverse traversal pattern used to print top-down derivation

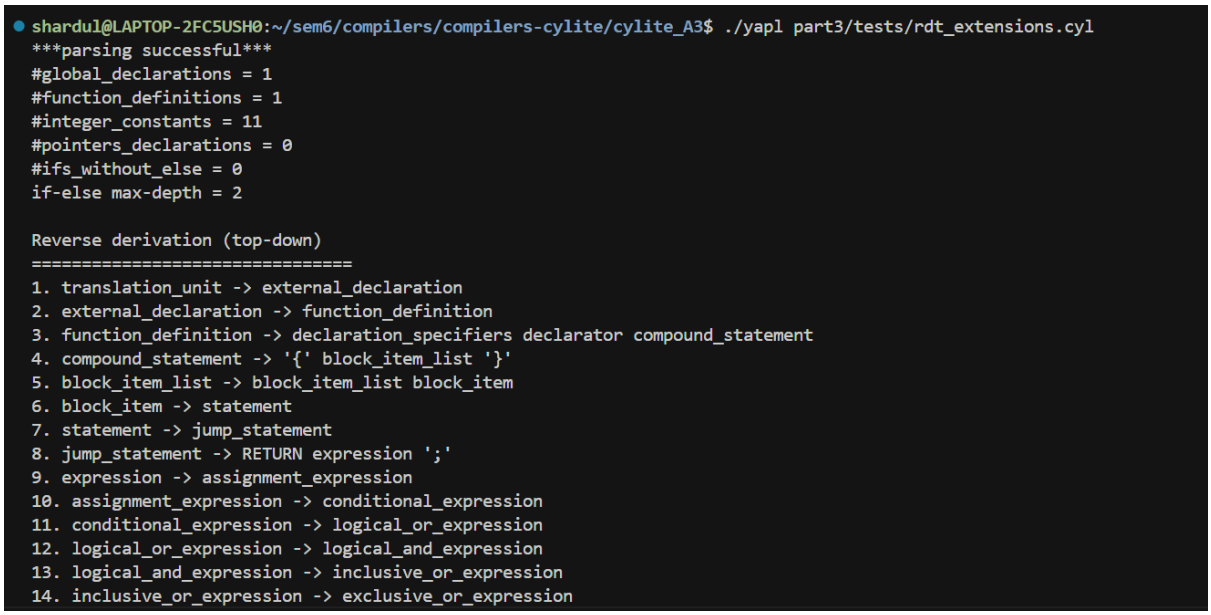
```
for (i = g_step_count - 1, j = 1; i >= 0; i--, j++)
{
    /* print: j. LHS -> RHS */
}
```

Conceptually, this is equivalent to reversing the reduction log. Since bottom-up parsers produce reductions from leaves toward the start symbol, the reverse sequence naturally reconstructs a derivation from start symbol toward terminals/nonterminals.

6.2 Terminal Output Mode

Representative command:

```
./yapl part3/tests/rdt_extensions.cyl
```



```
shardul@LAPTOP-2FC5USH0:~/sem6/compilers/compilers-cylite/cylite_A3$ ./yapl part3/tests/rdt_extensions.cyl
***parsing successful***
#global_declarations = 1
#function_definitions = 1
#integer_constants = 11
#pointers_declarations = 0
#ifs_without_else = 0
if-else max-depth = 2

Reverse derivation (top-down)
================================
1. translation_unit -> external_declaration
2. external_declaration -> function_definition
3. function_definition -> declaration_specifiers declarator compound_statement
4. compound_statement -> '{' block_item_list '}'
5. block_item_list -> block_item_list block_item
6. block_item -> statement
7. statement -> jump_statement
8. jump_statement -> RETURN expression ';'
9. expression -> assignment_expression
10. assignment_expression -> conditional_expression
11. conditional_expression -> logical_or_expression
12. logical_or_expression -> logical_and_expression
13. logical_and_expression -> inclusive_or_expression
14. inclusive_or_expression -> exclusive_or_expression
```

Figure 12: Part 3 terminal evidence: successful parse and reverse derivation output.

6.3 Visual Rendering using Graphviz (DOT and SVG)

To improve interpretability for long derivations, the parser also supports SVG generation through Graphviz:

```
./yapl part3/tests/rdt_extensions.cyl --svg part3/tests/
  rdt_extensions.svg
```

This produces:

- `part3/tests/rdt_extensions.svg`
- `part3/tests/rdt_extensions.svg.dot` (intermediate DOT graph)

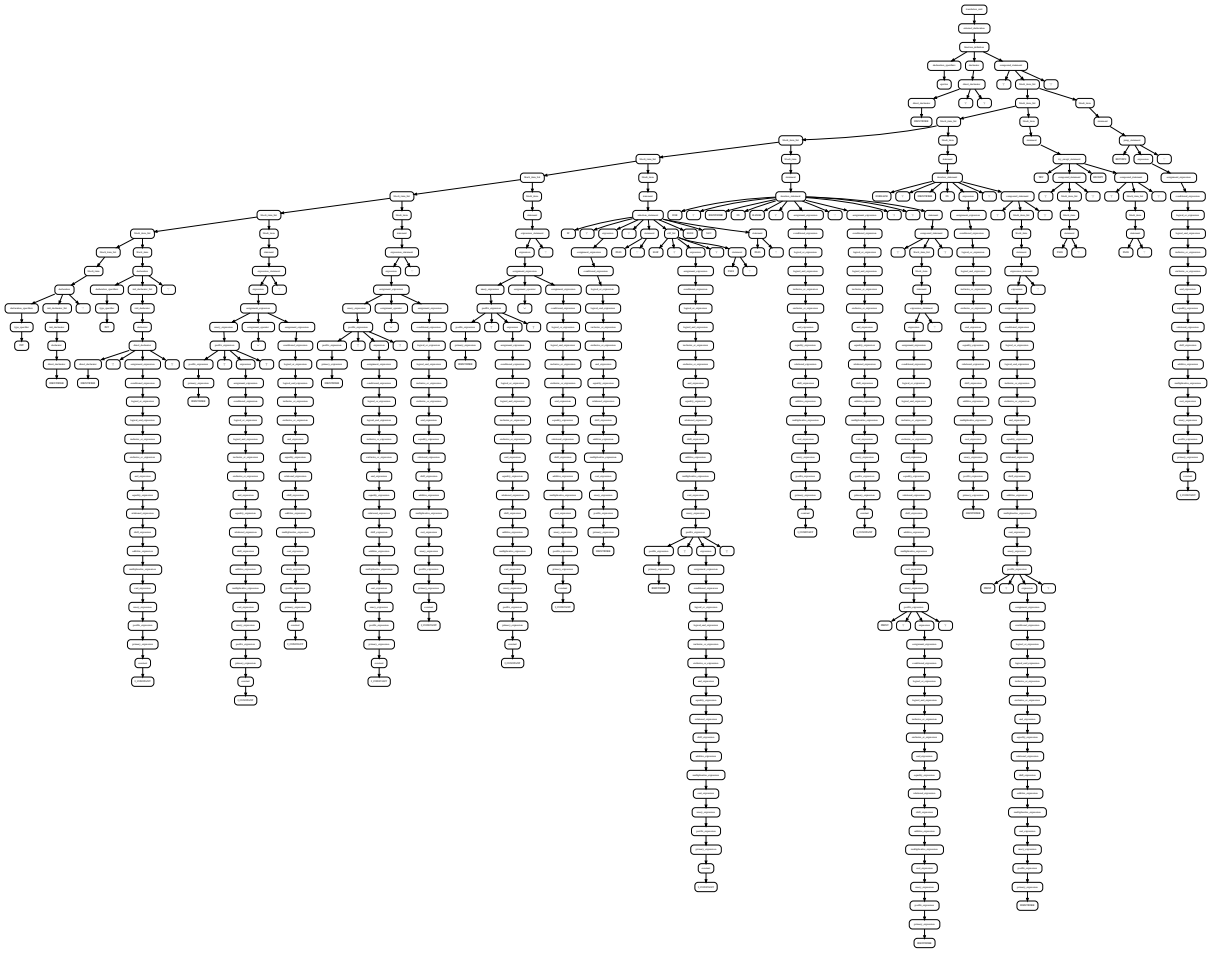


Figure 13: Graphviz visualization of reverse derivation structure.

7 Part 4: Parsing Table Output in Matrix Format

Part 4 required extraction of the LALR(1) table in matrix form. Instead of hand-copying transitions from `yapl.output`, an automated generator was implemented in Python (`part4/generate_parsing_table.py`).

7.1 Pipeline and Rationale

The implemented pipeline is:

1. Generate verbose parser report:
   `bison -v -d yapl.y`
2. Parse `yapl.output` programmatically.
3. Output CSV tables for ACTION and GOTO tables and a human-friendly matrix HTML view.

This approach guarantees reproducibility. Whenever grammar changes, tables can be regenerated with no manual reconciliation.

7.2 Produced Artifacts

The script generates four deliverables in `part4/`:

- `action_table.csv`
- `goto_table.csv`
- `parsing_table_summary.md`
- `parsing_table_matrix.html`

Representative run output:

```
python3 part4/generate_parsing_table.py yapl.output part4
Generated ACTION table: part4/action_table.csv
Generated GOTO table: part4/goto_table.csv
Generated summary: part4/parsing_table_summary.md
Generated HTML matrix: part4/parsing_table_matrix.html
```

From the generated summary report:

- Number of states: **519**
- ACTION columns: **107**
- GOTO columns: **80**

7.3 Matrix Design Notes

The HTML renderer intentionally combines ACTION and GOTO into a single wide matrix:

`State | ACTION terminals ...  | GOTO non-terminals ...`

To make a large LALR table usable, the following UX-level features were included:

- grouped section headers for ACTION and GOTO,
- sticky top headers,
- sticky state column,
- scroll container for high-column matrices.

Placeholder: Insert browser screenshot of parsing_table_matrix.html (single ACTION+GOTO matrix).

Figure 14: Single-matrix LALR(1) table view generated for Part 4.

7.4 Representative Implementation Snippets

Listing 5: State and ACTION/GOTO extraction strategy

```
state_re = re.compile(r"^State\s+(\d+)\s*$")
shift_re = re.compile(r"^\s+(\S+)\s+\[?shift, and go to state
   \s+(\d+)\]?")
reduce_re = re.compile(r"^\s+(\S+)\s+\[?reduce using rule\s
   +(\d+)\s+\(([^)]+)\)\]?")
```

```python

if symbol in nonterminals:
    add_cell(row_goto, symbol, target)
else:
    add_cell(row_action, symbol, f"s{target}")
```

The script also merges repeated entries in a cell as `value1 / value2`, preserving conflict visibility where applicable.

8 Part 5: Error Diagnostics

Part 5 focuses on actionable syntax diagnostics. The implemented behavior reports:

- exact line and column,
- the unexpected token actually seen,
- expected-token hints.

8.1 Diagnostic Architecture

The solution spans both lexer and parser:

1. Lexer tracks token start coordinates (`yytok_line`, `yytok_col`) through `YY_USER_ACTION`.
2. Parser uses a custom `yyerror()` path that prints structured syntax diagnostics.

Listing 6: Token-position capture in yapl.l

```
int yyline = 1;
int yycol = 1;
int yytok_line = 1;
int yytok_col = 1;

#define YY_USER_ACTION yytok_line = yyline; yytok_col = yycol
   ; update_position(yytext);
```

Listing 7: Parser-side diagnostic formatter in yapl.y

```
static void print_syntax_diagnostics(const char *s)
{
    const char *tok = "<end-of-input>";
    const char *expecting = NULL;

    if (yytext != NULL && yytext[0] != '\0') tok = yytext;
    printf("***parsing terminated*** [syntax error]\n");
    printf("line %d, column %d: unexpected token \"%s\"\n",
       yytok_line, yytok_col, tok);

    expecting = strstr(s, "expecting ");
    if (expecting != NULL) printf("expected: %s\n", expecting
        + 10);
}
```

8.2 Diagnostic Test Suite and Coverage

Invalid test programs were added under `part5/tests` and executed through `make part5`. Current suite:

- `diag_missing_semicolon.cyl`
- `diag_unexpected_else.cyl`

- diag_missing_semicolon_after_print.cyl
- diag_missing_closing_brace.cyl
- diag_bad_for_range_syntax.cyl
- diag_bad_try_except_syntax.cyl
- diag_invalid_character.cyl

8.3 Representative Output Samples

```
./yapl part5/tests/diag_missing_semicolon.cyl
***parsing terminated*** [syntax error]
line 4, column 5: unexpected token "x"
expected: ',' or ';'
```

```
./yapl part5/tests/diag_bad_try_except_syntax.cyl
***parsing terminated*** [syntax error]
line 5, column 9: unexpected token "pass"
expected: '{'
```

```
./yapl part5/tests/diag_invalid_character.cyl
***parsing terminated*** [syntax error]
line 4, column 5: unexpected token "@"
```

Placeholder: Insert terminal screenshot of make part5 diagnostics output.

Figure 15: Part 5 diagnostics run on intentionally invalid CYLite inputs.

9 Execution Instructions (Parts 3, 4, 5)

This section provides a direct, submission-ready command sequence.

9.1 Prerequisites

- Linux/WSL shell environment
- `bison`, `flex` (or `lex`), `gcc`
- `graphviz` (for SVG visualization in Part 3)

Quick checks:

```
bison --version
flex --version
gcc --version
dot -V
```

9.2 Build

```
cd cylite_A3
make clean
make
```

9.3 Run Part 3

```
make part3
```

This executes all Part 3 parser tests, writes SVG outputs in `part3/tests/`, and saves terminal outputs in `part3/tests/output_*.txt`.

9.4 Run Part 4

```
make part4
```

This regenerates `yapl.output` and all matrix artifacts in `part4/`.

9.5 Run Part 5

```
make part5
```

These cases are intentionally invalid. Non-zero parse exits are expected and are ignored by Make for demonstration. Each test also writes its parser output into `part5/tests/output_diag_*.txt`.

9.6 Optional Full Cleanup

```
make distclean
```

This removes parser-generated files and Part 3/Part 4 generated artifacts.

10 Conclusion

Parts 3, 4, and 5 extend the parser from a correctness-only front-end into a usable compiler component with traceability, observability, and practical feedback loops:

- Part 3 provides derivation introspection in both text and graph forms.
- Part 4 makes the LALR(1) table reproducible, auditable, and presentation-ready.
- Part 5 improves developer experience by turning generic syntax failure into contextual diagnostics.

Together with the completed conflict work from Parts 1 and 2, this delivers a robust Assignment-3 implementation aligned with both formal parser requirements and real debugging workflows.